VIETNAMESE - GERMAN UNIVERSITY
DEPARTMENT OF COMPUTER SCIENCE

Frankfurt University of Applied Sciences

Faculty: Computer Science and Engineering

Thesis topic: Improving Pneumonia Detection Accuracy with Object Detection and Image Preprocessing Techniques

Full name: Truong Dang Minh Khue

Matriculation number: 10421032

First supervisor: Dr. Nguyen Tuan Cuong

Second supervisor: Dr. Tran Hong Ngoc

BACHELOR THESIS

Submitted in partial fulfillment of the requirements for the degree of Bachelor Engineering in study program Computer Science, Vietnamese - German University, 2025

Binh Duong, Viet Nam

Declaration

I, Truong Dang Minh Khue, hereby affirm that I have completed this thesis on my own, under the supervision and guidance of Dr. Nguyen Tuan Cuong and Dr. Tran Hong Ngoc, after enrolling in the Bachelor of Computer Science program at Vietnamese-German University. This report does not provide any material that has been previously submitted in full or part to any degree, diploma or other qualifications and institutions.

Signature

Truong Dang Minh Khue
Computer Science Student
Vietnamese – German University

Acknowledgement

To begin with, I want to express my gratitude to my primary supervisor, Dr. Nguyen Tuan Cuong, for supporting and approving the thesis topic which I had planned to choose before. I appreciate his enthusiasm and the way he assists me to tackle problems step by step. In parallel, I would like to sincerely thank my secondary supervisor, Dr. Tran Hong Ngoc, for her encouragement and willingness to support during the thesis period. Furthermore, I would like to appreciate all my friends and colleagues, whose constant support, insightful discussions, and moral encouragement have helped me overcome many challenges. Their companionship has been invaluable throughout my research journey, and I am truly grateful for the positive influence they have had on my academic growth.

Abstract

Deep learning has transformed medical image segmentation, yet classical architectures like U-Net often face challenges with boundary ambiguity and inconsistent mask regions. Pneumonia lesion segmentation remains particularly difficult due to the scarcity of publicly available annotated datasets. In this context, the COVID-19 pandemic, which caused widespread morbidity and mortality, provides a relevant proxy, as COVID-19 infections can be viewed as a subcategory of pneumonia with similar radiological characteristics.

In this study, we develop and benchmark two pipelines for pneumonia-related lesion detection: a conventional U-Net and a hybrid U-Net integrated with a lightweight SAM-based refinement module (*U-Net + SAM-Adapter*). Multiple U-Net variants are first compared to identify the most stable backbone. The SAM-Adapter then refines coarse U-Net outputs using prompt-guided mask enhancement, improving mask coherence without retraining the full SAM.

Experiments demonstrate that the hybrid pipeline consistently outperforms the base U-Net in both IoU and Dice metrics, producing sharper lesion boundaries, reducing false positives, and stabilizing segmentation across challenging slices. These results highlight the potential of combining domain-specific architectures with foundation model adapters and illustrate a practical strategy for leveraging limited pneumonia data through COVID-19 CT scans.

Abbreviation

AI ..... Artificial Intelligence

DL ..... Deep Learning

CNN ..... Convolutional Neural Network

SMP ..... Segmentation Models PyTorch

ViT ..... Vision Transformer

SAM ..... Segment Anything Model

GT ..... Ground Truth

IoU ..... Intersection over Union

DSC ..... Dice Similarity Coefficient

AMP ..... Automatic Mixed Precision

GPU ..... Graphics Processing Unit

CPU ..... Central Processing Unit

BB ..... Bounding Box

FP ..... False Positive

FN ..... False Negative

TP ..... True Positive

TN ..... True Negative

LR ..... Learning Rate

UNet ..... Universal Network (Medical Segmentation Architecture)

EffNet ..... EfficientNet (Model Backbone Family)

DG ..... Domain Generalization

Table of Contents

List of Figures

Chapter 1

INTRODUCTION

1.1 Background

Deep learning has become a transformative tool in the medical field in recent years, enabling automation in diagnosis, anomaly detection, and image interpretation. By learning complex patterns from large-scale medical datasets, deep neural networks can assist clinicians in achieving faster and more accurate diagnostic outcomes. [1]

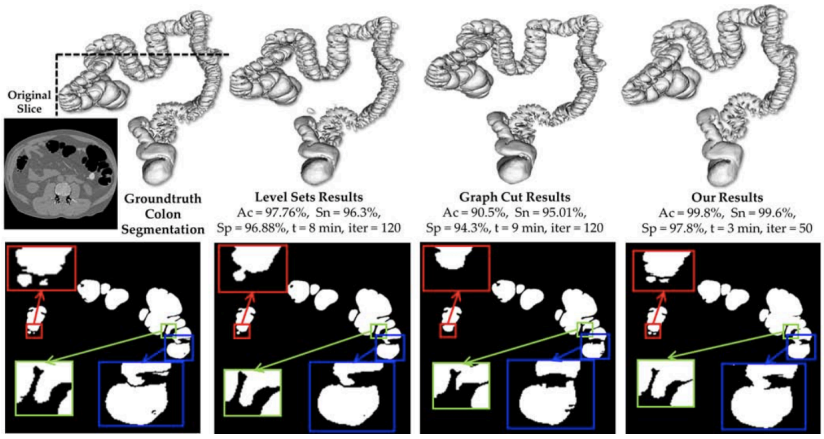


Figure 1.1: Medical Image Segmentation Example [1]

Among many various applications, **medical image segmentation** is particularly crucial. Anatomical features like organs, tissues, or lesions are identified and separated from scans using this procedure, which is critical for treatment monitoring, disease assessment, and surgical planning.

1.2 Motivation

Pneumonia lesion segmentation is vital to test lung and planning successful cures. Accurate detection of infections can help clinicians diagnose the disease early and track its course. However, there are quite *limited freely available* annotated pneumonia datasets. To address this, *COVID-19 CT scans*, which show radiological patterns similar to general pneumonia, act as a useful proxy for model building and evaluation.

Although the **U-Net** [2] architecture has become the standard for medical segmentation, it often struggles against complex textures, densed imagery or tight structural boundaries. Meanwhile, the **Segment Anything Model (SAM)** [3] provides strong general-purpose representations but lacks adaptation to medical imaging.

Meanwhile, the **Segment Anything Model (SAM)** [3] provides strong general-purpose representations but lacks adaptation to healthcare imaging.

1.3 Contributions

- **Benchmarking U-Net variants:** Multiple U-Net structures with various encoder-decoder configurations were evaluated comprehensively for pneumonia lesion segmentation.
- **Proposed SAM-Adapter:** A fine-tuning module that adapts SAM for medical segmentation, focusing on infected lung regions [4].
- **Hybrid Semantic Segmentation pipeline:** A framework that combines U-Net and SAM-Adapter to improve U-Net prediction accuracy and boundary precision. [5]
- **Comprehensive evaluation:** Extensive studies were conducted using publicly available COVID-19 CT scans as a proxy for pneumonia, validating accuracy gains and generalization performance.

Chapter 2

Related Works

2.1 U-Net and its Variants

The original U-Net, introduced by Ronneberger et al. (2015) [6], laid the foundation for modern medical image segmentation. Its encoder–decoder structure with skip connections allows effective feature extraction and spatial detail preservation, making it ideal for biomedical imaging.

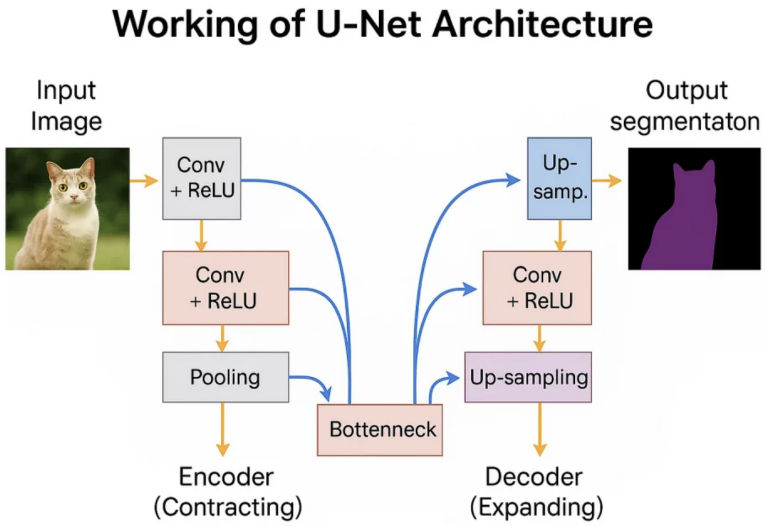


Figure 2.1: U-net Architecture [2]

U-Net's symmetric architecture balances localization with semantic correctness, allowing for efficient performance even with minimal input:

$$\hat{y} = f_{\text{dec}}\big(f_{\text{enc}}(x)\big),$$

where f_{enc} and f_{dec} represent the encoder and decoder functions, respectively.

2.1.1 U-Net and its Variants

In this study, multiple U-Net variants were implemented using the **Segmentation Models PyTorch (SMP)** [7] library for benchmarking:

DeepLabV3+ with EfficientNet-B3 / MiT-B1

SegFormer with EfficientNet-B3 / MiT-B1

U-Net++ with EfficientNet-B3

These configurations allow us to compare convolution-based and transformer-based decoders for pneumonia spot segmentation.

2.2 Vision Transformers and SAM

Vision Transformers (ViTs) [8] are an alternative to CNNs that use self-attention to represent long-range dependencies between image patches. They provide different benefits for global context modeling and semantic understanding.

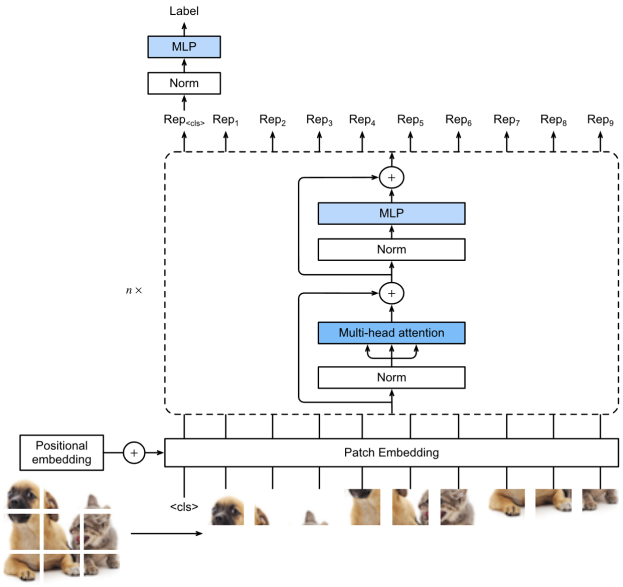
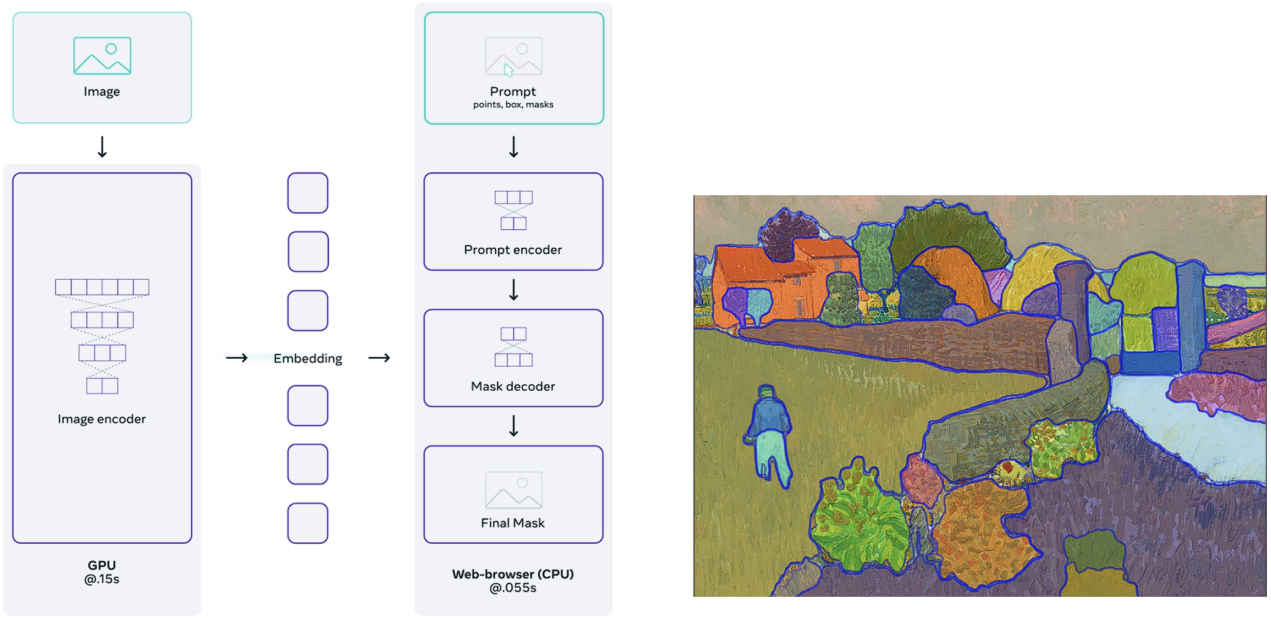


Figure 2.2: Vision Transformer Architecture [8]

2.2.1 Segment Anything Model (SAM)

The Segment Anything Model (SAM), developed by Meta AI is a large-scale ViT trained on *billions of masks* for general-purpose segmentation. SAM generates object masks without task-specific training, demonstrating strong generalization across diverse domains.



(a) SAM Architecture (b) SAM Example

Figure 2.3: Segment Anything Model Architecture [9]

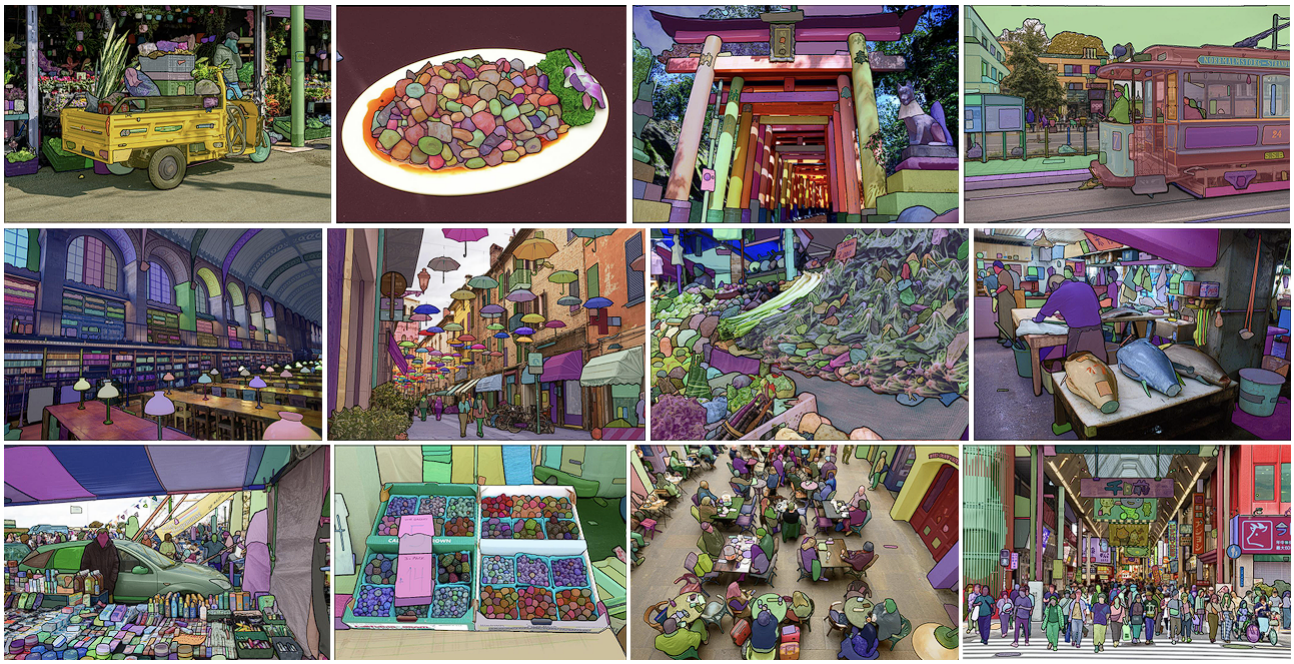


Figure 2.4: Training Masks for SAM [3]

However, using SAM directly to medical images frequently yields inferior findings due to domain gaps in texture, intensity, and structure. Recent studies, including *SAM-Med2D* and *MedSAM* [4], fine-tuned SAM for medical datasets, achieving enhanced organ and lesion segmentation.

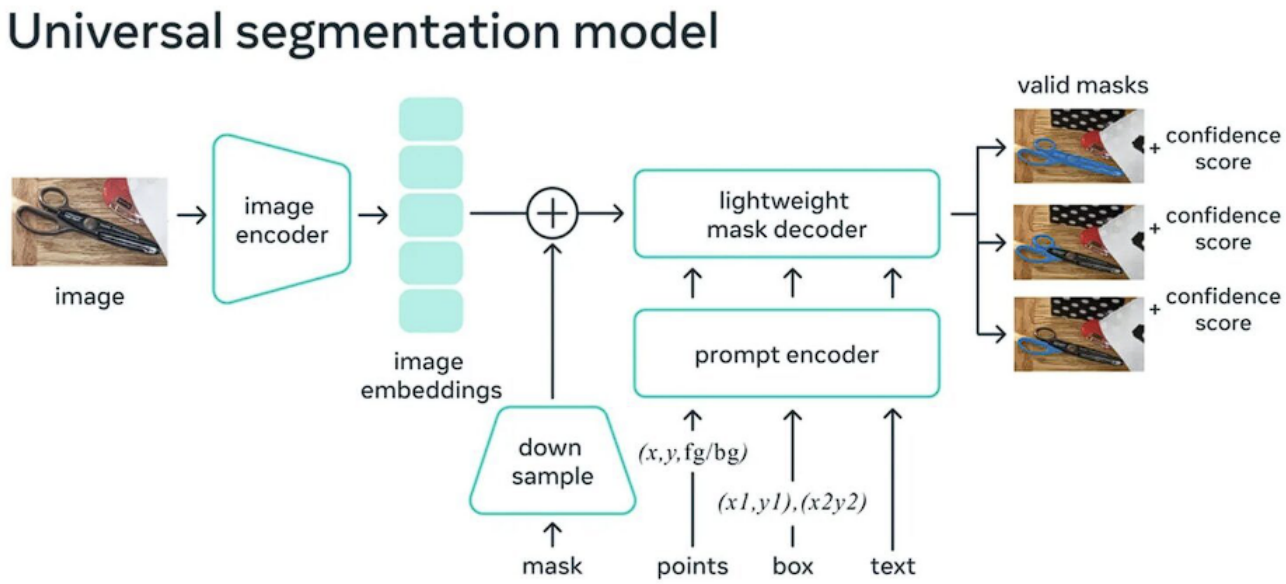


Figure 2.5: SAM architecture [3]

SAM comprises three main components:

Image Encoder

Transforms the input image into dense embeddings (vectorization) to capture global and local features, allowing generalization across domains.

Prompt Encoder

Accepts user-provided or algorithmic prompts that guide segmentation:

- **Point prompts:** specify target or background pixels.
- **Box prompts:** define rough bounding regions.
- **Mask prompts:** provide prior segmentation for refinement.

These are embedded in the same latent space as image tokens, enabling cross-attention between modalities.

Mask Decoder

Image and prompt embeddings are fused together using transformer layers to build exact masks that take advantage of both spatial and semantic clues. amid medical contexts, this process allows for accurate zone demarcation even amid noisy or restricted monitoring.

2.2.2 Adapter: Better Fine-tuning and Domain Adaptation

It is computationally heavy due to fully fine-tune SAM due to its large parameter count.

To mitigate this, an **adapter-based fine-tuning** approach is employed [10]. Lightweight adapter modules are inserted into transformer layers while keeping SAM's pre-trained weights frozen [4].

This design offers two key benefits:

- **Efficiency:** Only a small number of parameters are trained, cutting down computation and time.
- **Domain Accuracy:** The adapters tailor SAM's representations to medical textures, improving segmentation precision without losing generalization.

Overall, SAM-Adapter provides a practical and efficient way to adapt large foundation models for lung infection segmentation.

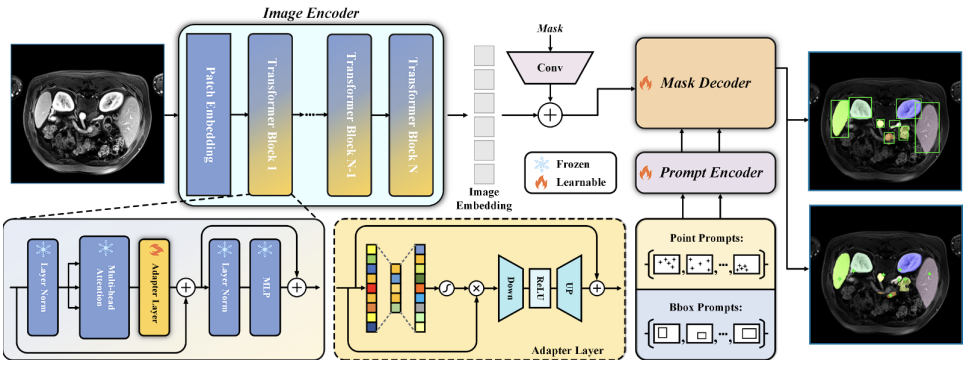


Figure 2.6: SAM Adapter Architect [4]

2.2.3 Hybrid Approaches

Hybrid techniques combine segmentation models (e.g., U-Net) with foundation models (e.g., SAM or CLIP) to achieve exact localization using multi layers features and semantic prior knowledge from large-scale transformers.

$$\hat{y} = f_{\text{SAM}}\big(f_{\text{U-Net}}(x)\big),$$

where $f_{\text{U-Net}}$ provides structural context and f_{SAM} refines semantic boundaries.

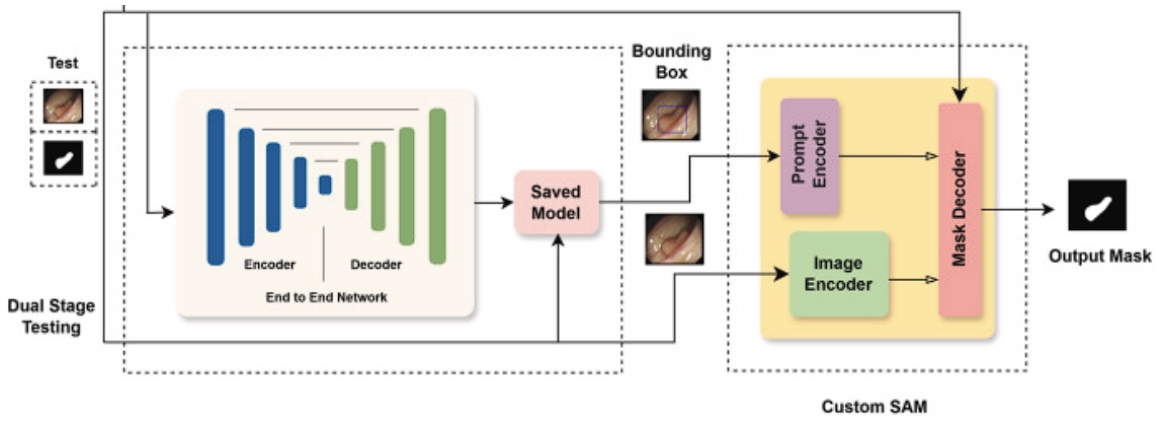


Figure 2.7: Hybrid U-Net + SAM-Adapter Pipeline Example. [5]

This research will compare the result masks predicted by these 2 pipelines:

- **Only U-net**
- **U-net and SAM-Adapter:** U-net will create masks that work as prompts for SAM-Adapter to refine.

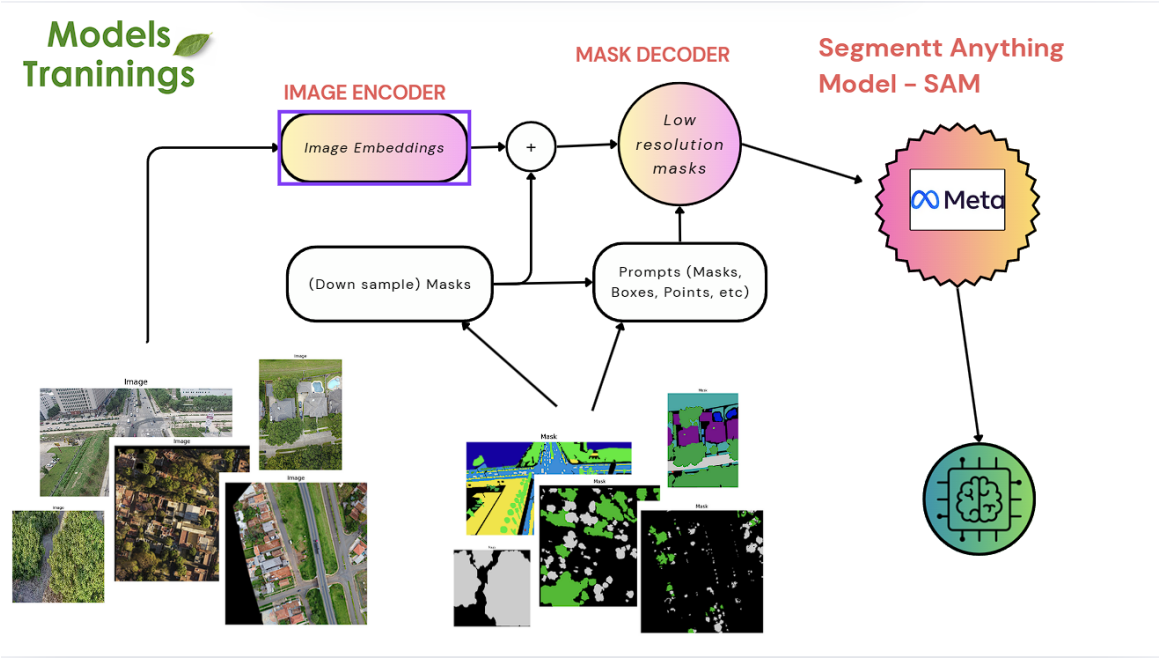


Figure 2.8: U-NET and SAM pipeline Example

Chapter 3

METHODOLOGY

3.1 Overall Pipeline Overview

3.1.1 Training Setup

U-Net Architectures

Several U-Net variations were used to evaluate encoder-decoder systems [7]. The encoder creates the feature extraction backbone, while the decoder manages the up-sampling and skip-connection strategies.

Encoders: The ResNet34 (resnet), EfficientNet-B3 (effnet), and Swin Transformer (mit) backbones were utilized to represent convoluted and transformer-based features.

Decoders: Popular model types like U-net++, SegFormer, Deeplabv3plus, can be found and applied with ease thanks to SMP.

Base SAM Architecture

A Base-SAM pipeline was implemented to establish a baseline for comparison against the proposed SAM-Adapter [9].

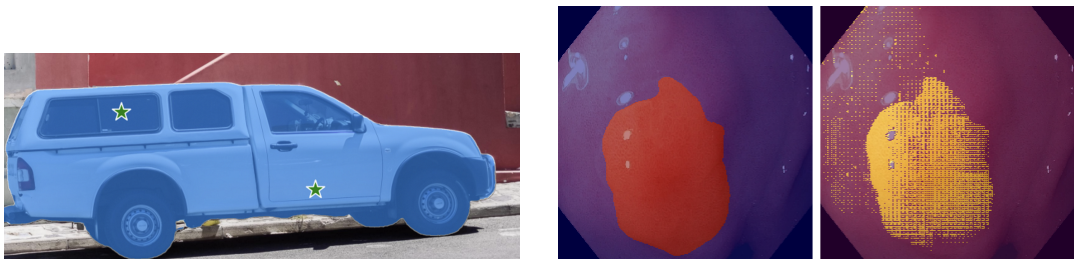


Figure 3.1: Base SAM examples [3]

SAM-Adapter Architecture

As of this work, no existing libraries directly support SAM with adapters, thus the implementation was manually constructed following Meta AI's original codebase [3] [4].

Base Model: The original SAM architecture with a Vision Transformer (ViT) backbone served as the frozen feature extractor.

Adapter Modules: Bottleneck layers and attention adapters were put together with transformer layers to boost feature representations in medical images.

Optimizer and Scheduler

Each model was optimized using `AdamW` [11] with safe parameter grouping for encoder, decoder, and segmentation head components. Two scheduler strategies were evaluated: `OneCycleLR` [12] and `MultiStepLR` [13].

AdamW's decoupled weight decay improves regularization stability compared to traditional Adam.

The AdamW optimizer:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1-\beta_1) g_t, \\ v_t &= \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \\ \hat{m}_t &= \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \\ \theta_{t+1} &= \theta_t - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda\, \theta_t \right), \end{aligned} \tag{3.1}$$

where θ_t denotes the parameters at step t, g_t the gradient, β_1, β_2 the exponential decay rates, η the learning rate, ϵ a numerical stability constant, and λ the weight decay coefficient.

Loss functions

Loss compositions were empirically chosen to balance pixel-level precision and structural overlap [14].

- **BCEDiceLoss** – balancing pixel-wise precision and region overlap,
- **BCETverskyLoss** – controlling false positives/negatives via (α, β) weights,
- **FocalLoss** – emphasizing hard examples,
- and for SAM-Adapter: **FocalDiceIoULoss** – combining region and boundary consistency.

Early Stopping

Early stopping was employed to prevent over-fitting by monitoring validation metrics [15]. For the U-Net pipeline, a custom score was defined as:

$$\text{Score} = -0.25 \times \text{avg\_train\_loss} - 0.25 \times \text{avg\_val\_loss} + 0.5 \times \text{mean\_IoU}, \tag{3.2}$$

and training was terminated once this score exceeds the parameter 'patience'. And for the SAM-Adapter pipeline, early stopping was decided purely on loss optimization, with lower loss indicating better model performance.

Data augmentation

Includes standard resizing, random rotations, flips, and light-based transformations [16]. The datasets for U-net pipeline are split into 80% training and 20% validation.

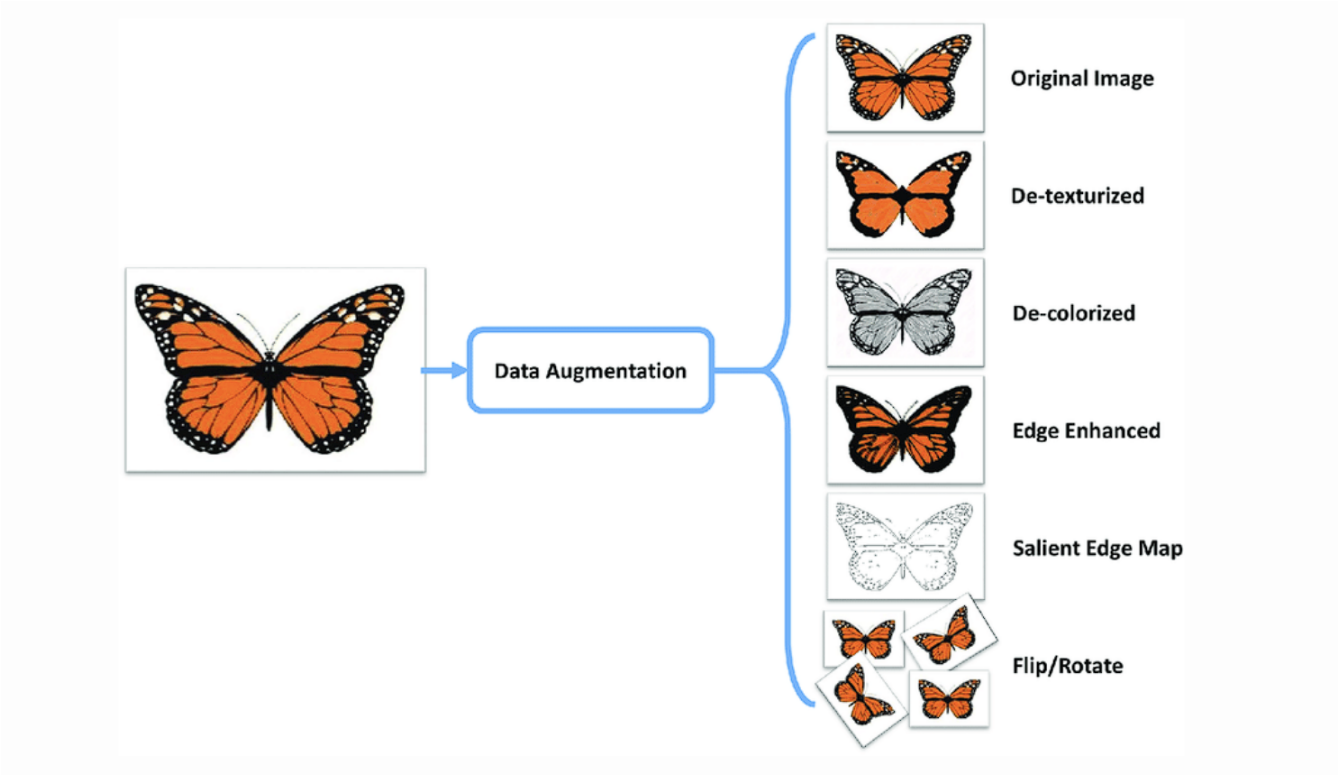


Figure 3.2: Data Augmentation [16]

All of the above functions can be easily added by using Albumentations [17].

Data augmentation was applied to increase dataset diversity, improve model generalization, and reduce overfitting.

3.1.2 Training Strategies with Multiple Datasets

To increase the performance of models, I use multiple datasets from different sources. Two common considered strategies:

Table 3.1: Comparison of training strategies with multiple datasets

Strategy	Description, Pros & Cons
Fine-tuning	Pretrain on one dataset, then sequentially fine-tune on others [18]. ***Pros:*** Leverages knowledge from a large source dataset; often achieves high performance on target. ***Cons:*** May suffer catastrophic forgetting; requires sequential access and careful LR tuning; risk of overfitting or confusion if datasets conflict.
Multi-dataset Training	Concatenate datasets (e.g., using `ConcatDataset`) and train simultaneously [19]. ***Pros:*** Efficiently uses all data; helps learn shared representations. ***Cons:*** Does not explicitly enforce domain-invariant learning; performance may degrade under domain shift.

Comment: For this study, **concatenating multiple datasets** proved more effective for handling diverse patient scenarios and improving generalization.

3.1.3 Web-based Deployment

A web application was developed using **Streamlit** [20] to facilitate real-time medical segmentation with multiple U-Net models. Key features include:

- **Multi-model selection:** Users can choose among different U-Net variants for inference.
- **On-the-fly prediction:** Upload images and receive segmentation masks immediately.
- **Visualization:** Overlay predicted masks on original images for easy inspection.

- **Lightweight interface:** Minimal setup required, suitable for rapid prototyping and demonstration purposes.

This deployment enables non-technical users to interact with trained models, compare outputs across different architectures, and assess segmentation quality without writing code.

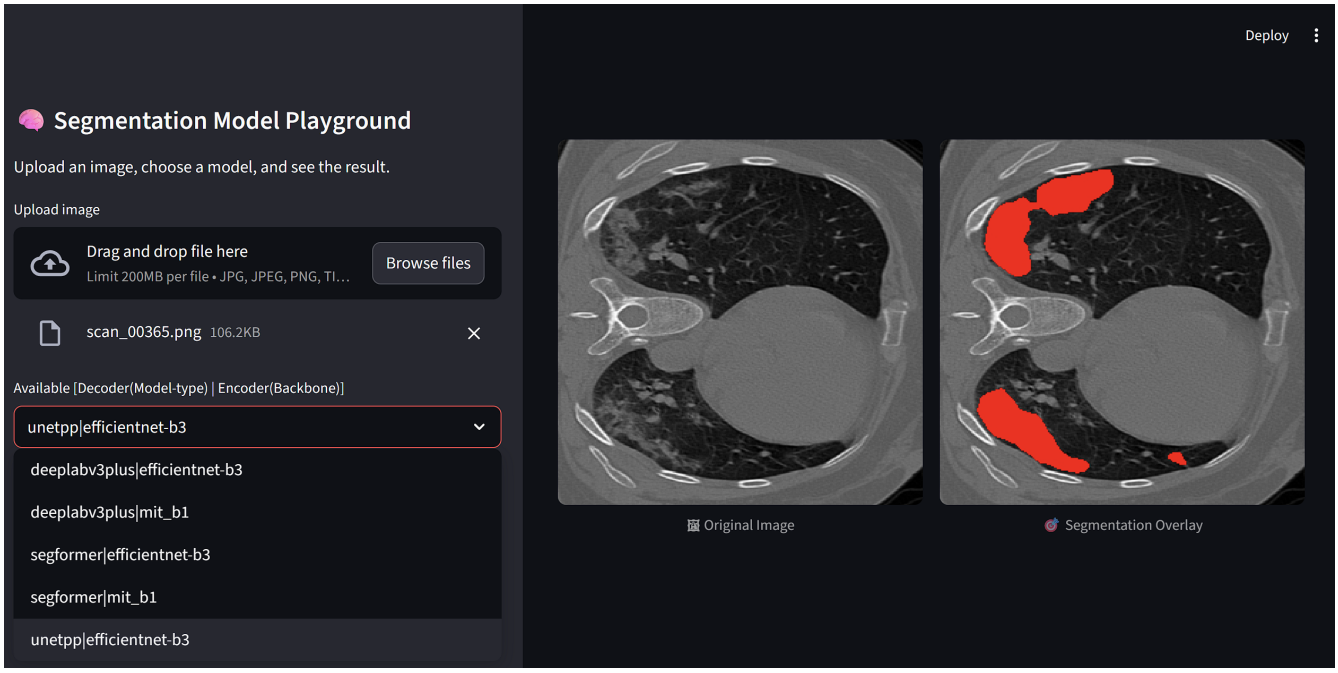


Figure 3.3: Streamlit App

This interface demonstrates the model's real-world applicability, bridging research and clinical utility.

Chapter 4

Experimental Setup

4.1 Dataset

Experiments were carried out using two COVID-19 CT scan datasets, which serve as benchmarks for medical image segmentation and include diverse imaging scenarios that support model generalization.

Training and Validation Datasets Overview

- **COVID-19 CT scans:** Comprises four folders—'CT scan images', 'infected lung masks', 'lung masks', and 'both infected and lung masks'—each containing 20 NIfTI (.nii) volumes [21].
- **Lung CT Nodule/Lesion Segmentation:** Contains 843 MB of images and masks with numerous unique cases for pneumonia detection [22].

Pre-processing Medical Data

Medical images are provided in various formats, including .PNG, .JPG, .NII, and .DCM. For our pipeline, all images are converted to .PNG.

For the *COVID-19 CT scans* dataset [21], the primary format is **.NII**, which stores volumetric CT data as 3D arrays. Each volume represents a full lung scan, from

which axial slices can be extracted. Two main challenges arise when dealing with X-ray data:

- **Redundancy:** Converting NIfTI volumes to PNG generates hundreds of slices per patient, many of which are highly similar, introducing redundant data.
- **Background regions:** Many slices contain large black areas that are irrelevant for model training and visualization.
- **Unclear foreground:** The target regions in some X-ray slices are faint or low-contrast, making them difficult to delineate.

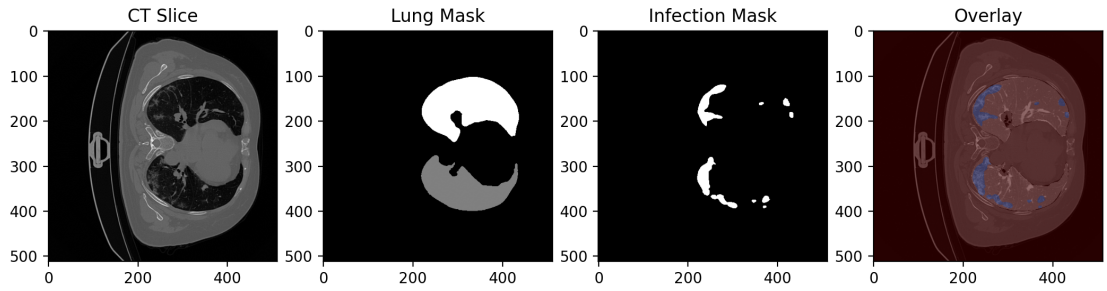


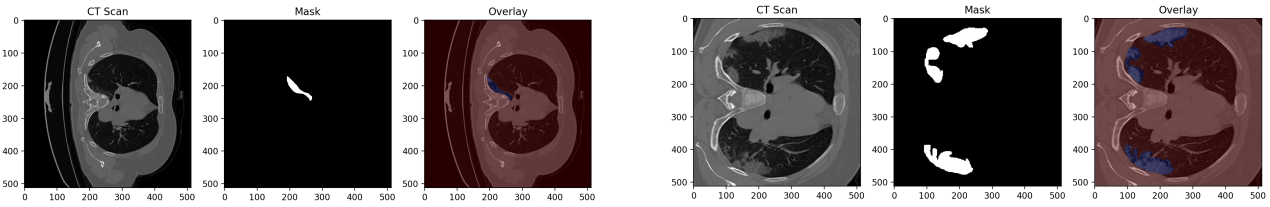
Figure 4.1: COVID Samples

If unaddressed, these issues can cause:

- **Reduced performance:** Model performance improves with more data only when it is clean and diverse; *redundant slices can lead to over-fitting*.
- **Longer training time:** Excessive, non-informative slices increase training time without benefiting the model.

To mitigate data redundancy and irrelevant background, *lung masks* were used to define **a region of interest (ROI) around the lungs** [**ROI**], ensuring the model focuses on informative areas. *Duplicate slices* and approximately *(80 percent)* of slices containing *only background* in the infection masks were excluded to reduce noise and prevent overfitting. All remaining slices were converted to *8-bit format*, *normalized*,

and mapped using the *bone colormap*, which preserves the relative intensity of lung structures while maintaining visibility of subtle infection patterns.



(a) First example (b) Second example

Figure 4.2: Post-processed Medical Images

While some slices are well-processed and clearly show lung structures, others remain extremely challenging to interpret due to poor contrast, unusual intensity distributions, or incomplete scans.

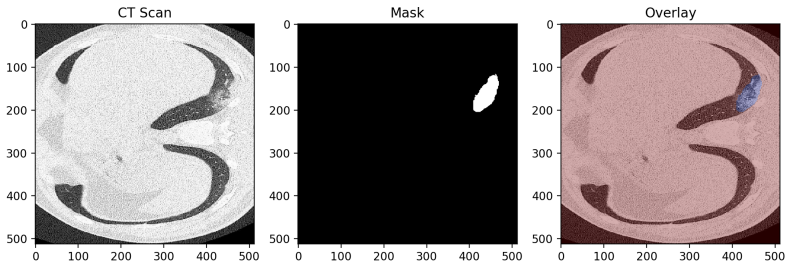


Figure 4.3: Difficult-to-interpret example

Furthermore, certain slices are misleading or of low quality, which could negatively impact model performance.

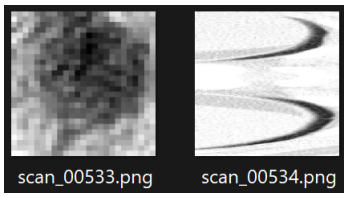


Figure 4.4: Low-quality or misleading slices

Although manual inspection and removal of undesired slices is possible, it is time-consuming and inefficient. A better approach would be to employ a classification model to filter slices that do not contain lung regions **(anomaly data)**. For the sake of study complexity, this approach was not implemented in the current pipeline.

As a result, some inherent limitations in data quality may lead to slightly reduced performance of the trained models, particularly on challenging or low-quality slices.

Preprocessing functions for training

To ensure consistent training and fair comparison across datasets, the following preprocessing steps were applied:

- **Resizing:** All images were resized to 256×256 pixels. This dimension was chosen as a trade-off between computational efficiency and detail preservation, and it also matches the input constraints of models like SAM.
- **Normalization:** Each image channel was standardized to have zero mean and unit variance, following ImageNet normalization statistics.
- **Tensor Conversion:** Images and masks were converted to PyTorch tensors to enable GPU acceleration and efficient backpropagation.
- **Augmentation:** Random horizontal/vertical flips, small rotations, and brightness adjustments were used to improve robustness while avoiding unrealistic distortions common in medical imagery.

All of the above functions can be easily added by using Albumentations [17].

4.1.1 DataLoader and Concatenation

Multiple datasets were concatenated using PyTorch's `ConcatDataset` [23] to form a unified training set. This technique allows the model to learn shared representations across domains, resulting in increased robustness to domain transitions. However, pure concatenation does not guarantee domain invariance.

In some cases, we want local features of targeted objects to be more focused on. For example, a Vietnamese banana to general bananas. That is why I implement

something called **domain-based generalization**, where selected data representing the target domain are prioritized and effectively increased during training.

In this study, annotated pneumonia masks are scarce. To mimic a practical pipeline, the **COVID-19 CT scans dataset** was divided into three smaller subsets, simulating data from different sources and capturing **general lung infection features**. The main data will give specific attributes, showing what a local hospital may prioritize.

4.1.2 Selecting Datasets

COVID-19 CT scans dataset was divided into three smaller subsets. The first two data will be used for training, and **the last one is for evaluation.** See Chapter 5 for more inferences details.

The *domain-based data* I selected was **Lung CT Nodule/Lesion Segmentation** [22] samples, which was repeated two times during training. Although this introduces a slight bias toward the focused domain, it is beneficial in practice, provided the data are clean, moderate in size, and representative.

Note: While concatenation increases data diversity, caution must be exercised to prevent dataset imbalance and overfitting to the dominant source domain.

4.1.3 Implementation Details

Framework: PyTorch with `segmentation-models-pytorch` [7]. **Hardware:** NVIDIA RTX 3060 GPU with CUDA and cuDNN.

Key Hyperparameters:

- **Learning rate:** 1×10^{-4} — chosen to balance stable convergence and sufficient weight updates.
- **Batch size:** 8 — small enough to fit GPU memory with high-resolution images, yet large enough for stable gradient estimates.

- **Epochs:** 200 — provides adequate iterations for refinement without overfitting.
- **Early stopping patience:** 7 — to prevent unnecessary training once performance plateaus.
- **workers:** a hyperparam for Dataloader. We can max it out, as long as it is suitable for your GPU.

4.1.4 Training Procedures

Key aspects of the training pipeline include:

- **Automatic Mixed Precision (AMP):** Training leverages `torch.cuda.amp` for faster computation and reduced memory usage [24].
- **Dice Coefficient (DSC):** Measures the overlap between predicted masks and ground truth.
- **Intersection over Union (IoU):** Evaluates the pixel-wise agreement between prediction and target.
- **Precision:** Proportion of correctly predicted positive pixels over all predicted positives.
- **Recall:** Proportion of correctly predicted positive pixels over all actual positives.
- The total loss can be computed:

$$\mathcal{L}_{total} = 2 \cdot \mathcal{L}_{Focal} + \mathcal{L}_{Dice} + 1.0 \cdot \mathcal{L}_{MaskIoU}$$

Metrics [25] were computed per-image and averaged across the validation set. The IoU score was also used to guide **early stopping**.

4.2 SAM-Adapter Training Strategy

Training of the SAM-Adapter is a bit special [4]. It needs to be conducted in a staged manner to efficiently adapt the large pretrained SAM model to the COVID lesion segmentation domain.

Step 1: Prompt processing

Prompts, as explained before, are *guidance* for SAM to track objects. It can still predict without prompts, but the results can be extremely bad. Prompts are strictly required for this pipeline to work properly.

The prompts can be bounding boxes, points or masks, or all of those above. In the **training pipeline**, each image receives only **a single bounding box prompt**. In addition, **point prompts** are generated *randomly* within the regions where the predicted mask differs from the ground truth. This simulates *interactive user correction* and **encourages the model to focus on difficult areas**, enabling the decoder to iteratively refine the segmentation masks.

For tasks like lesion detection, these two types of prompts might be enough, giving balanced guidance for SAM. Prompt masks are usually unnecessary, and it this case, they might give too much information to SAM. So not only removing prompt masks encourages SAM to self-learn through light advice, it also simplifies the training process and reduces computational overhead.

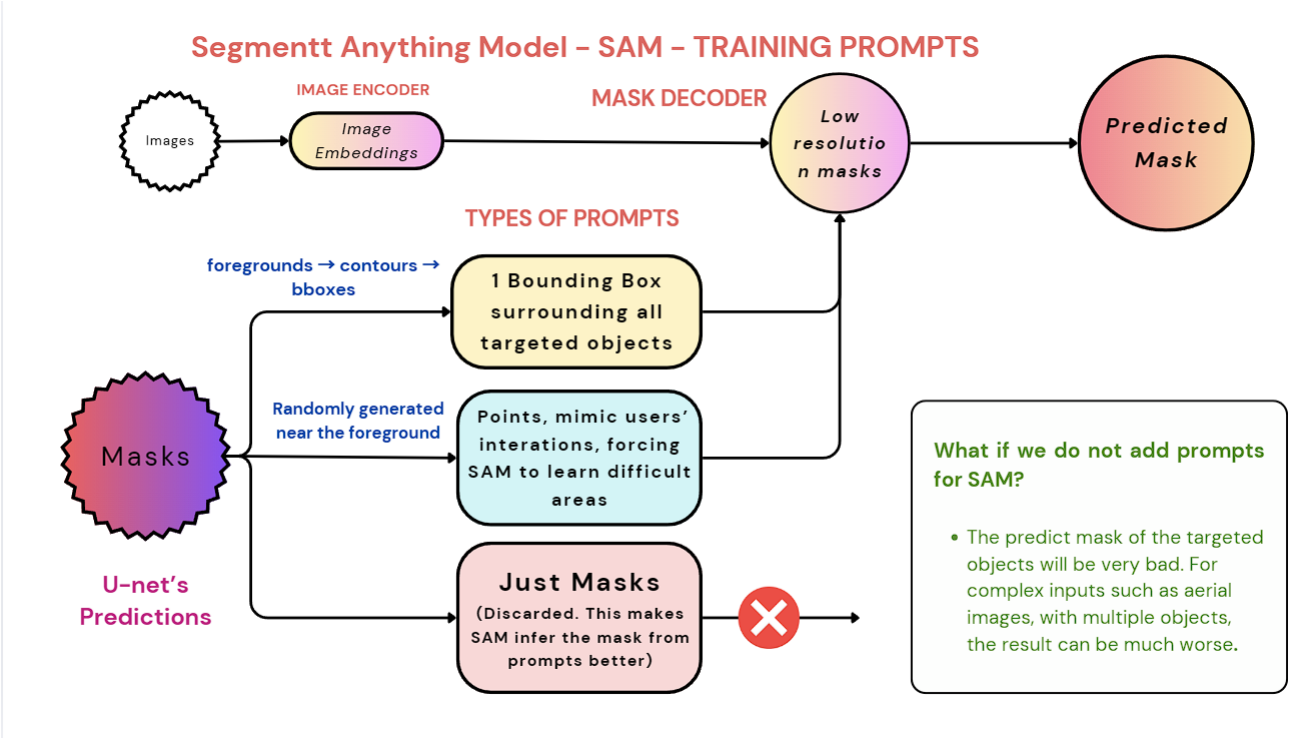


Figure 4.5: SAM Prompts Explaination

Note: It is expected that the quality of prompts decides the output masks of SAM. That is only partly true, because:

- SAM-Adapter is surprisingly robust to coarse guidance.
- Even when bounding boxes or points are 'loosen', the fine tuned model can correctly identify the relevant foreground region. This will be clearer in the results below.

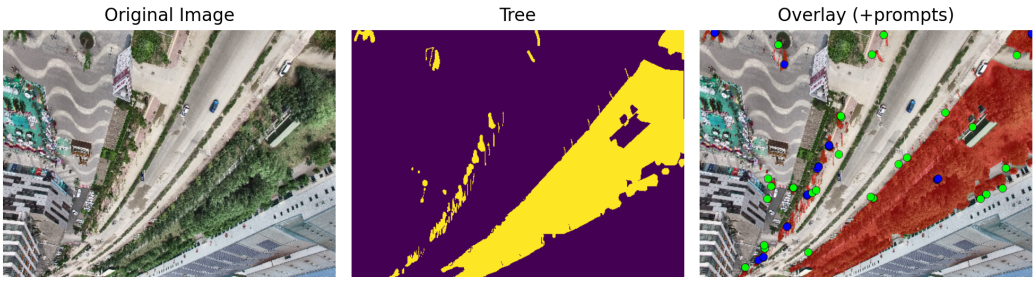


Figure 4.6: Prompt Examples

Step 2: Adapter Fine-tuning

Only the lightweight adapter modules within the image encoder are updated, while the rest of SAM's parameters remain **frozen**. Input images are encoded, prompts (points or boxes) are generated, and the mask decoder makes first predictions. The training loss is then calculated and back propagated through the adapters. This step allows the model to rapidly absorb niche, specific features at a cheap computational cost.

Step 3: Full Decoder Training with Iterative Prompting

The image embeddings are detached, and all other parts of the model (prompt encoder and mask decoder) are trained while keeping the image encoder frozen. Incorrect predictions are used to generate new prompts iteratively, allowing the decoder to refine masks through self-correcting feedback. This stage improves segmentation accuracy while leveraging the strong pretrained representations of SAM.

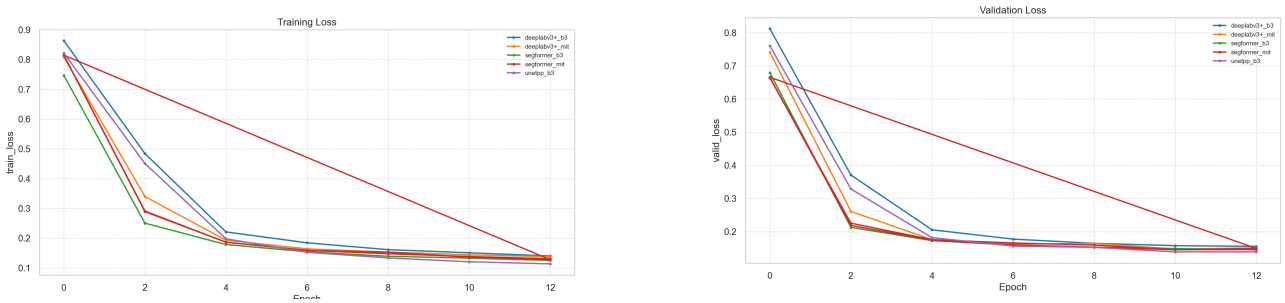
Overall, this two-stage strategy balances *training efficiency* and *domain-specific adaptation*, enabling high-quality mask refinement without full fine-tuning of SAM.

Note: When in inference mode, randomly generated points are no longer useful since this time, actual users will interact with the inputs. The users, alongside with 1 bounding box, will guide the model to predict, if it need to be fine tuned.

Chapter 5

Results and Discussion

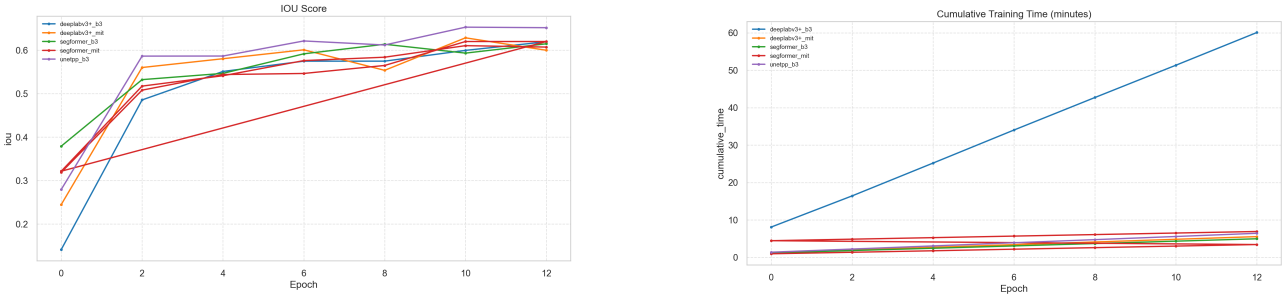
5.1 U-net Quantitative Results



(a) Train Loss results (lower is better)

(b) Valid Loss results (lower is better)

Figure 5.1: Quantitative comparison of training and validation loss for U-Net variants.



(a) IoU results (higher is better)

(b) Cumulative training time (lower is better)

Figure 5.2: IoU performance and cumulative training time of U-Net variants.

For X-ray datasets, *a reduction in training and validation loss* indicates that the model is improving its ability to *predict the correct locations* of lesions. However, loss alone does not quantify how closely the predicted masks align with the ground

truth. This alignment is captured by the Intersection over Union (IoU) score. In *imbalanced* datasets such as X-rays, where background pixels (black) vastly outnumber foreground pixels (gray, representing lesions), models may achieve low loss by predominantly predicting background pixels. *The IoU metric*, however, provides a more precise assessment of the model's capability to *accurately identify the affected regions of the lungs.*

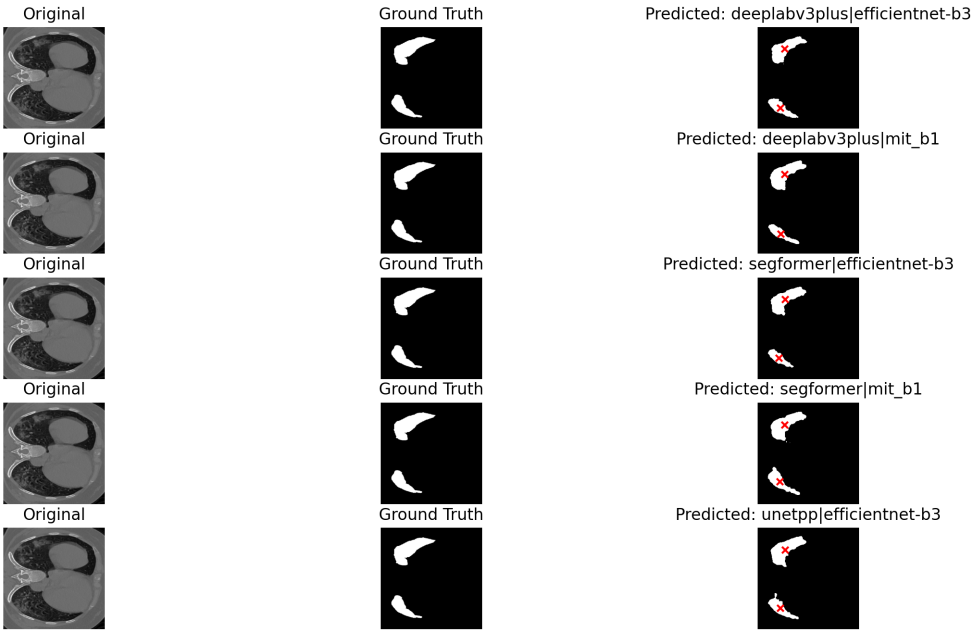


Figure 5.3: U-net Variants Prediction (Easy Case)

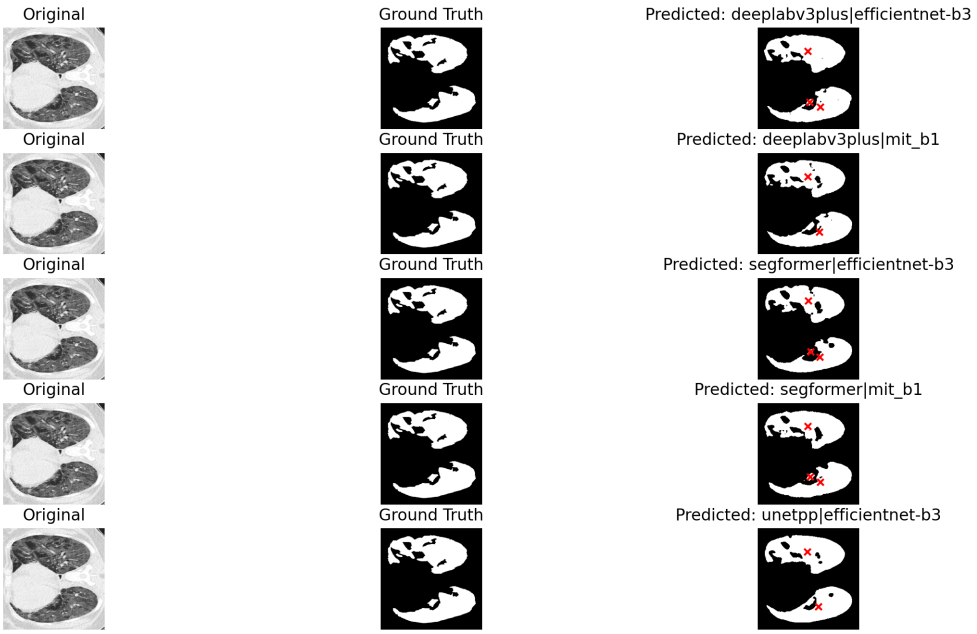


Figure 5.4: U-net Variants Prediction

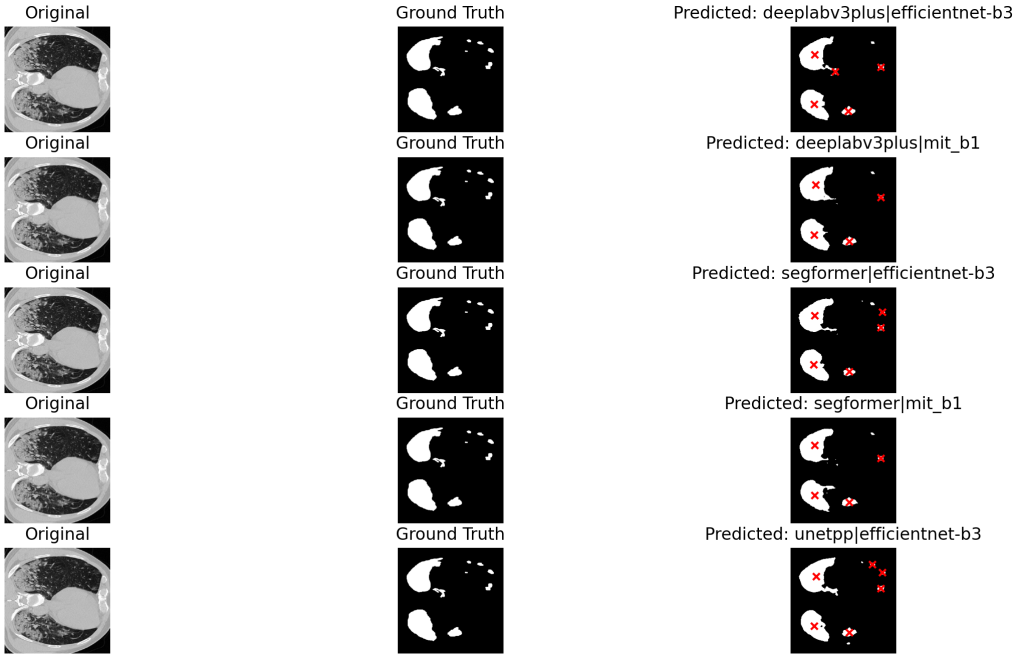


Figure 5.5: U-net Variants Prediction (Hard Case)

Among all variants, U-Net++ with EfficientNet-B3 backbone achieved the best trade-off between accuracy and efficiency. **Therefore, I selected it as the base model for subsequent SAM-Adapter experiments below.**

5.2 Model Ensemble Methods

In this work, we explore two ensemble strategies for combining predictions from multiple models: **Soft Voting** and **Weighted Voting**.

5.2.1 Soft Voting

Soft voting combines the predicted probabilities from all models by simply averaging [26]. Given n models with predicted probabilities p_i, the final prediction P_{final} is computed as:

$$P_{\text{final}} = \frac{1}{n} \sum_{i=1}^{n} p_i$$

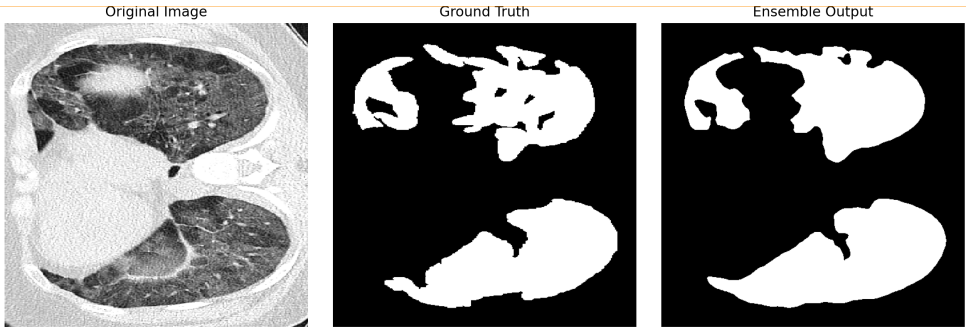


Figure 5.6: Visualization of Soft Voting

5.2.2 Weighted Voting

Weighted voting extends soft voting by assigning each model a weight w_i, often based on validation performance [26]. Here I just give each model weight based on their best IoU score. The final prediction is then:

$$P_{\text{final}} = \frac{\sum_{i=1}^{n} w_i \cdot p_i}{\sum_{i=1}^{n} w_i}$$

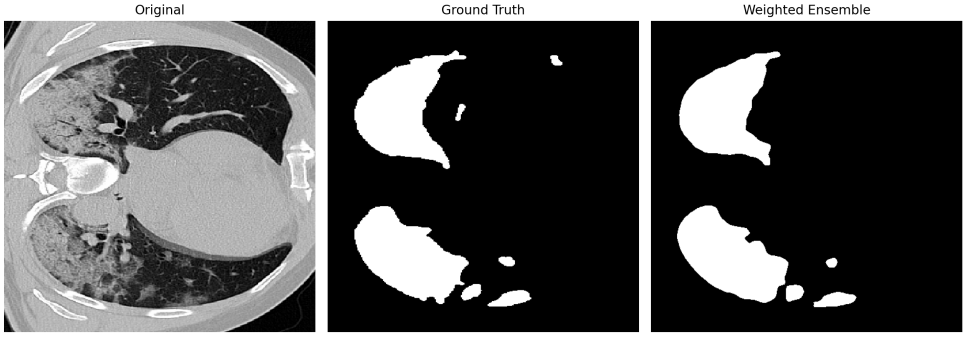
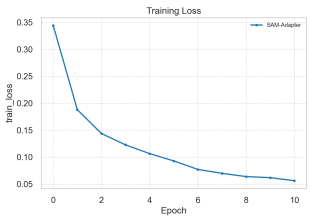


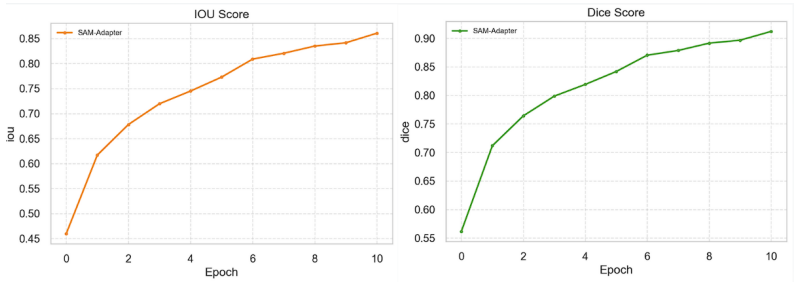
Figure 5.7: Visualization of Weighted Voting

Both ensemble strategies provide a mechanism to aggregate model confidence, leading to smoother and more stable predictions across test samples.

5.3 SAM-Adapter Quantitative Results



(a) SAM-Adapter Training Loss (Lower is better)



(b) SAM-Adapter Score (Higher is better)

Figure 5.8: Quantitative comparison of training and validation loss for U-Net variants.

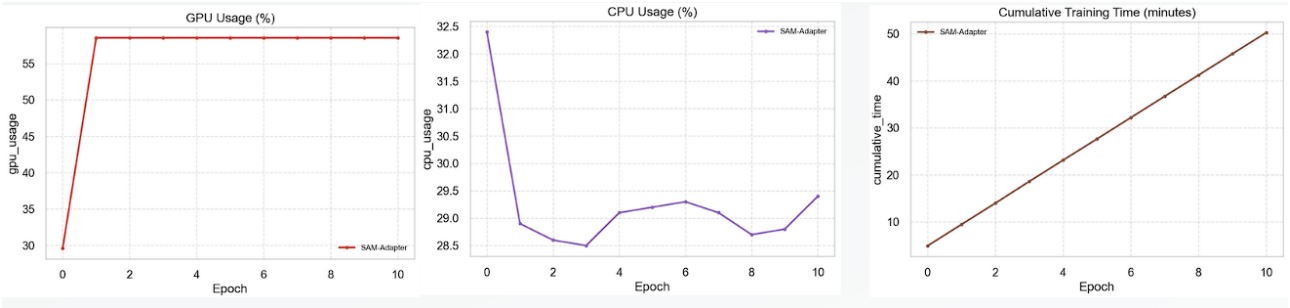


Figure 5.9: Internal Usage and Time Spent for Training

> **Note on DataLoader configuration:** *Due to compatibility issues between PyTorch's DataLoader and the Windows multiprocessing spawn method, setting num_workers > 0 occasionally caused unexpected import errors and dataset access conflicts. To ensure stable and reproducible data loading, all experiments in this study were performed with num_workers=0. While this configuration increases loading time, it guarantees correctness and prevents multiprocessing-related crashes.*

This is the performance results of SAM-Adapter pipeline. The loss rapidly decreased, while the scores increases to above 0.85. This is a splendid result, demonstrate strong convergence and stable optimization of SAM itself.

As expected, even though we are working with a literal ViT -based architecture, applying an adapter helps a lot with time spent to train. The internal resources usage

is also low. Training 256x256 datasets with purerly SAM can lead to OOM very easily.

Unfortunately that the Dataloaders' workers have to be set to 0 because of multi-processing conflict, so the cumulative time increases by a lot. But for a large model like SAM, training 10 epochs for only 50 minutes is a great success, and Adapter has prove its worth.

5.3.1 SAM-Adapter vs Base SAM

We will now compare the practical results from a fine tuned SAM-Adapter with an 'innocent' SAM2 that has never seen the Pneumonia datasets.

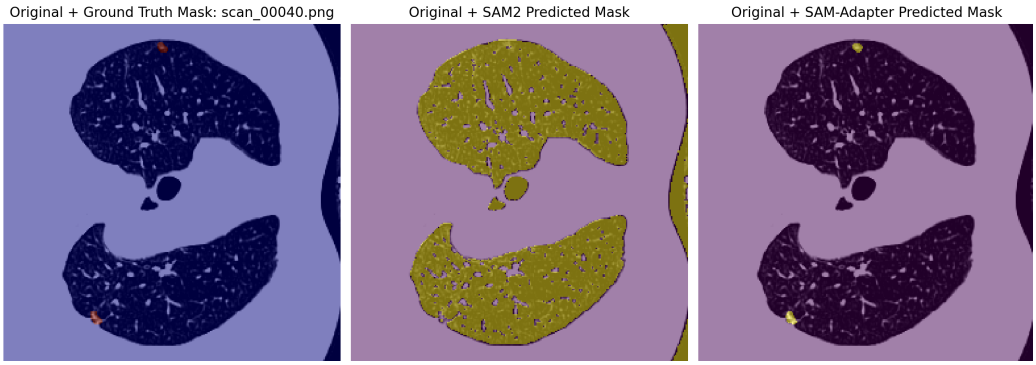


Figure 5.10: Base SAM and fine-tuned SAM-Adapter (set 1).

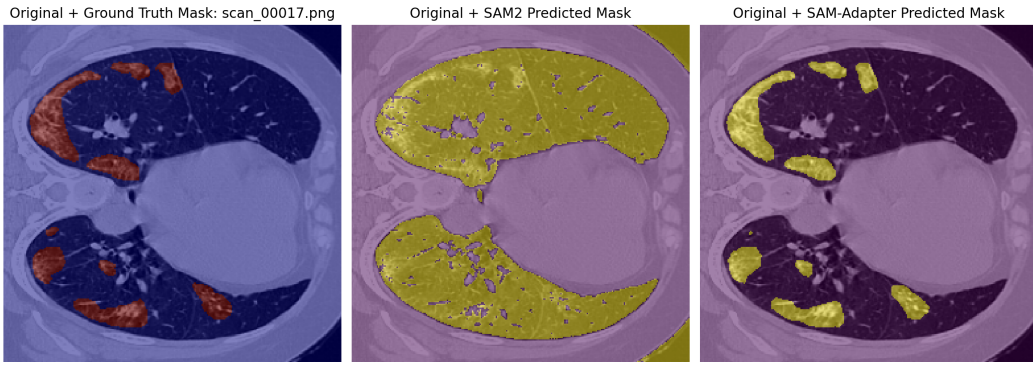


Figure 5.11: Base SAM and fine-tuned SAM-Adapter (set 2).

Note that every SAM models require **prompts**, or **guidance**, as inputs, so when using in more familiar data, Base SAM 2 can almost give correct predictions. But in this case, where the lesions are hard to notice, it is clear that the fine-tuned SAM-

Adapter outperforms the Base SAM and exhibits strong adaptation to domain-specific textures.

The qualitative comparisons highlight SAM-Adapter's superior localization and boundary delineation, especially under challenging illumination or low-contrast conditions.

Summary: The experimental results confirm that integrating SAM-Adapter with a strong U-Net++ backbone can potentially, substantially improves segmentation accuracy and boundary precision. To test this hypothesis, let's go to the next chapter and answer this study's main goal: Will SAM-Adapter enhance U-net pipeline accuracy?

Available Github Code: [27]

Chapter 6

Ablation Studies

6.1 Pipeline Comparison: U-Net vs U-Net + SAM-Adapter

To quantify the contribution of the SAM-Adapter, two segmentation pipelines were evaluated:

1. **U-Net Only:** Standard U-Net++ (EfficientNet-B3 encoder), trained for 22 epochs.
2. **U-Net + SAM-Adapter:** U-Net generates initial masks, refined by the fine-tuned SAM-Adapter using one bounding-box prompt (U-Net: 12 epochs, SAM-Adapter: 10 epochs).

Note: Both pipelines were trained and evaluated under identical conditions for a fair comparison.

Note: Just a reminder, from previous chapter, it is decided that the chosen U-net variant comprises of *unet++ and effnet-b3 encoder*.

6.1.1 Workflow to calculate metrics

To fairly compare the results, some pre-works are:

1. Firstly, train the U-net model (for 12 epochs)
2. Secondly, fine tune the SAM-Adapter (for 10 epochs)

3. Then feed the generated masks from U-net predictions as prompts for SAM.

4. The SAM-Adapter, which converts masks into 1 box as guidance, will now refine the input masks. Those output will be stored in a folder for later usage **(important)**.

5. Then, we continue to train the U-net model from the last checkpoint (like the first step - for another 10 epochs) so that it can predict on pair with SAM-Adapter.

6. This 22 epochs U-net model will also predict masks and store them in another folder, thus, create a only U-net pipeline.

7. Finally, we use these 3 folders: (1) the original masks; (2) the refined masks (U-net + SAM-Adapter); and (3) the U-net-only masks.

Note: All the masks have to be resized to match each other.

6.2 IoU and Dice Score Comparison

IoU measures how much the predicted region overlaps with the ground truth, while Dice score measures their similarity using the *harmonic mean* of their intersection and total areas.

In practice, higher IoU and Dice scores mean the model's predicted masks align more closely with the true object regions — indicating more accurate segmentation. Lower scores mean the model either misses parts of the object (under-segmentation) or includes too much background (over-segmentation).

```
Model
U-net + SAM-Adapter    0.855118
Unet                   0.739636
Name: IoU, dtype: float64
```

(a) Mean IoU score between 2 pipelines

```
Model
U-net + SAM-Adapter    0.897466
Unet                   0.830659
Name: Dice, dtype: float64
```

(b) Mean Dice score between 2 pipelines

Comparison plots

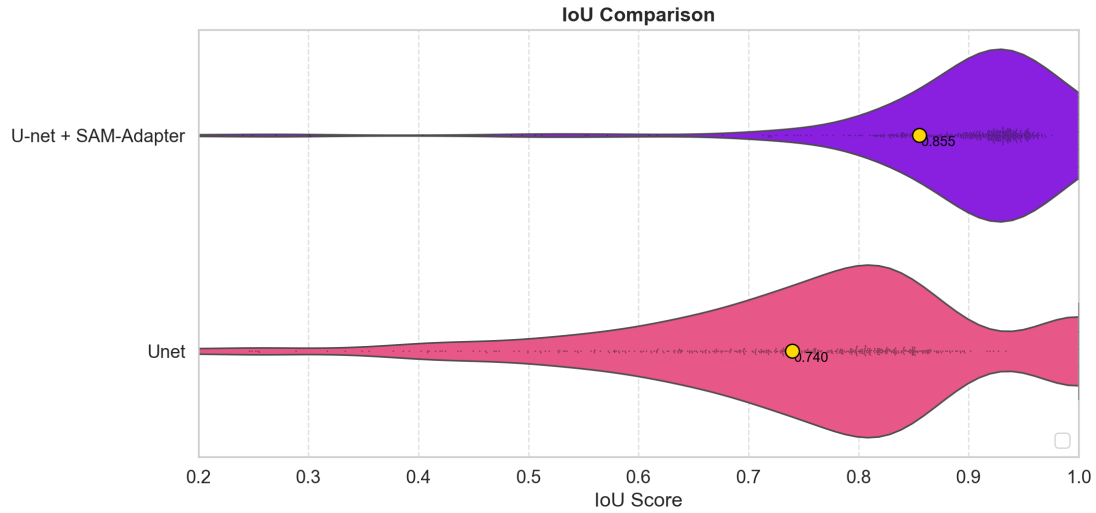


Figure 6.2: Violin plots of Iou scores

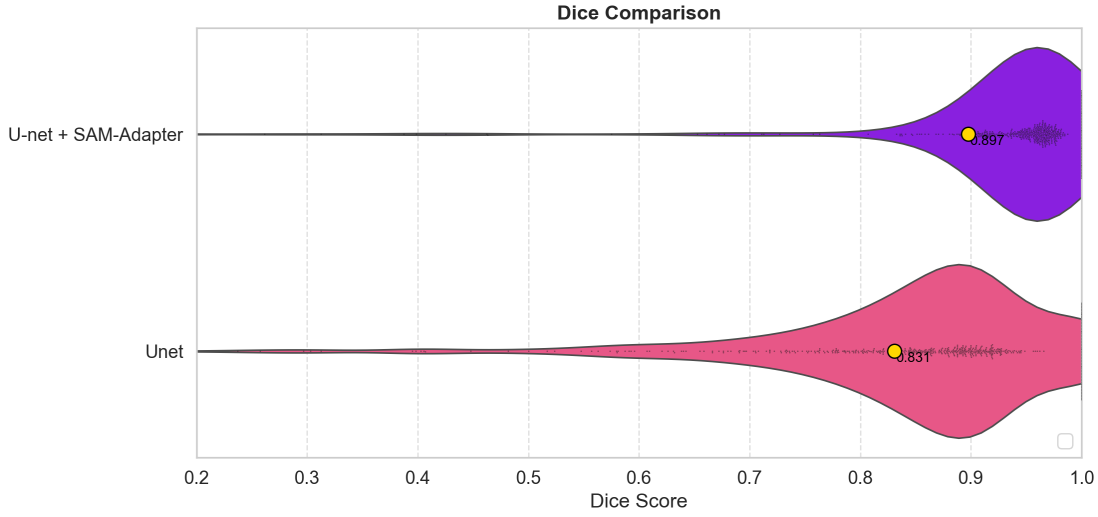


Figure 6.3: Violin plots of Dice scores

As shown in the violin plots [28], the UNet model exhibits a *wider score distribution* with *several outliers*, indicating **less consistent** segmentation performance across samples. In contrast, the SAM-Adapter results are more compact and concentrated around higher IoU and Dice values, suggesting **better stability and overall accuracy**.

6.3 Practical Comparison

To complement quantitative metrics, qualitative visualizations were conducted to illustrate how SAM-Adapter refines the segmentation boundaries.

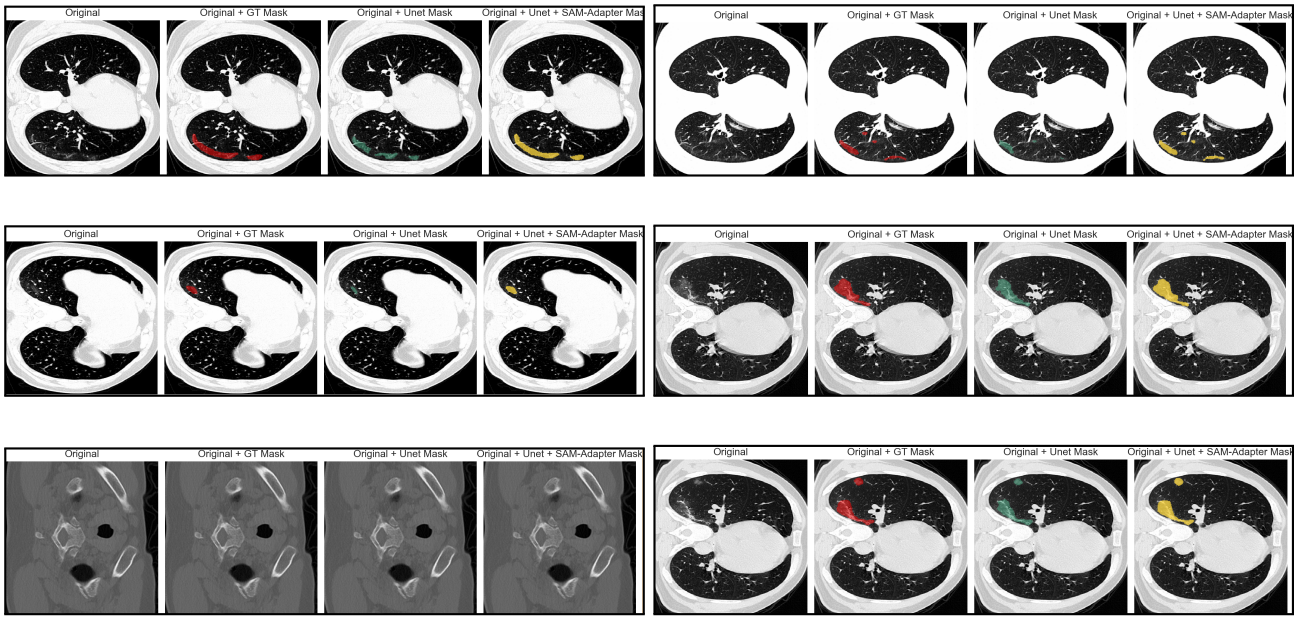


Figure 6.4: Qualitative comparison between U-Net and U-Net + SAM-Adapter across six test cases.

Each column pair shows results from the same input sample. *Red masks* are the *Ground-truth masks*, the *Green* ones are from the *U-net only pipeline*, and the *golden* ones are predictions of *U-net and SAM-Adapter* pipeline.

Despite being trained for a similar number of epochs, the hybrid pipeline achieves both higher accuracy and faster convergence.

6.4 Practical Comparison - Special Observations

In U-net and SAM-Adapter pipeline, SAM receives re-created masks from U-net, which are then converted into bounding boxes as prompts.

I mentioned before that the quality of these prompt masks are partially matter. As shown below, even when the U-Net's predicted masks produce sub-optimal bounding boxes, SAM-Adapter can still recover accurate segmentation through its learned contextual understanding:

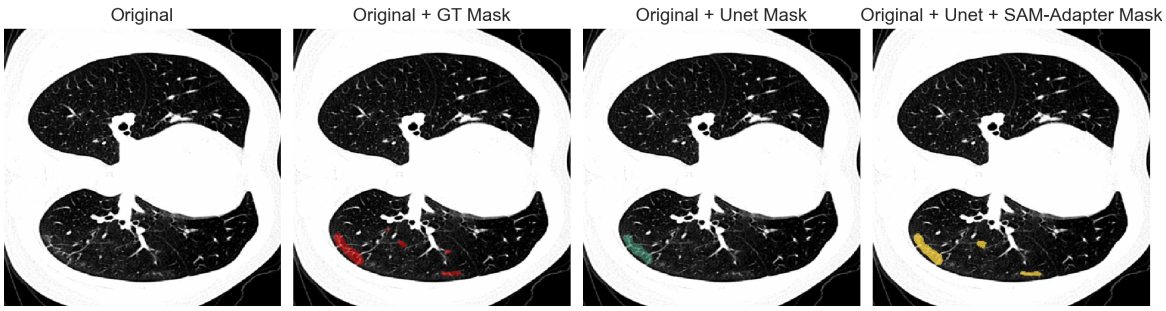


Figure 6.5: SAM-Adapter correctly identifies the target despite a misleading prompt box.

In the first showcase, although the U-net fails to fully reconstruct masks with complete foreground coverage, it still, provides valuable target localization. That is enough for U-net + SAM-Adapter pipeline to create nearly accurate masks.

Now in this case, the U-net has failed to even localize all targeted objects. This leads to a much smaller prompt bounding box. Nevertheless, the U-net + SAM-Adapter pipeline can still catch the missing ones. This is because SAM-Adapter treats the prompts as guidance rather than strict constraints.

Summary

The ablation results demonstrate that integrating U-net model with SAM-Adapter substantially improves segmentation accuracy, stability, and boundary precision. While the baseline U-Net produces solid initial masks, the adapter consistently enhances them — sharpening edges, reducing false positives, and aligning more closely with anatomical structures.

This synergy shows that the SAM-Adapter functions not merely as a post-processing module, but as an intelligent refinement layer with fast training time that extends U-Net's contextual reasoning.

In essence, the hybrid U-Net + SAM-Adapter pipeline combines the domain-specific learning of U-Net with the adaptive generalization of SAM, offering a powerful direction for future medical segmentation research.

Note: my code got conflicted with Window's multiprocessing workflow so I had to set Dataloader's workers to 0 in SAM-Adapter pipeline. Recommended using number of workers > 0 for full blast in speed. **Available Github Code:** [27]

Chapter 7

Conclusion and Future Work

While this study demonstrates the strong potential of combining U-Net and SAM-Adapter for medical image segmentation, several promising research directions remain open.

First, the current approach is entirely **supervised**, relying on pixel-level annotated data. Future work may explore **semi-supervised** or **weakly supervised** learning settings, where SAM's prompt-based generalization could compensate for limited annotations.

Second, incorporating **domain adaptation** strategies could enhance generalization across different imaging modalities or organ types, leveling up robustness in unseen clinical environments. Improving on the training inputs is upmost crucial. Adding a classification model for fast ROI is recommended.

Third, beyond bounding-box prompts, future systems could integrate **multi-modal cues** — such as textual or radiological metadata — to provide richer contextual understanding.

Finally, a more unified training framework that jointly optimizes both U-Net and the Adapter, rather than training them sequentially, may unlock deeper synergy between task-specific feature learning and SAM's foundation-level representations.

Overall, this study establishes a strong foundation for developing adapter-based **transfer learning frameworks** that bridge the gap between general-purpose vision

models and specialized medical imaging applications, especially in Pneumonia - COVID19 lesion detection.

References

[1] Risheng Wang, Tao Lei, Ruixia Cui, Bingtao Zhang, Hongying Meng and Asoke K. Nandi. *Medical Image Segmentation Using Deep Learning: A Survey*. URL: https://arxiv.org/pdf/2009.13120.

[2] Capa Learning. *Understanding U-Net Architecture in Deep Learning*. URL: https://capalearning.com/2025/05/29/understanding-u-net-architecture-in-deep-learning/.

[3] Meta AI Research, FAIR. *Segment Anything*. URL: https://arxiv.org/pdf/2304.02643.

[4] Junlong Cheng, Jin Ye, Zhongying Deng, Jianpin Chen, Tianbin Li, Haoyu Wang, Yanzhou Su, Ziyan Huang, Jilong Chen, Lei Jiang, Hui Sun, Junjun He, Shaoting Zhang, Min Zhu, Yu Qiao. *SAM-Med2D*. URL: https://arxiv.org/pdf/2308.16184.

[5] Radiful Islam, Rashik Shahriar Akash, Md Awlad Hossen Rony, Md Zahid Hasan. *SAMU-Net: A dual-stage polyp segmentation network with a custom attention-based U-Net and segment anything model for enhanced mask prediction*. URL: https://www.sciencedirect.com/science/article/pii/S2590005624000365.

[6] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. *U-Net: Convolutional Networks for Biomedical Image Segmentation*. URL: https://arxiv.org/pdf/1505.04597.

[7] Segmentation Models Community. *Segmentation Models Pytorch's Documents*. URL: https://smp.readthedocs.io/en/latest/encoders.html.

[8] Wikipedia community. *Vision transformer*. URL: https://en.wikipedia.org/wiki/Vision_transformer#Overview.

[9] Meta AI Research, FAIR. *Segment Anything*. URL: https://segment-anything.com.

[10] Zhe Chen, Yuchen Duan, Wenhai Wang, Junjun He, Tong Lu, Jifeng Dai, Yu Qiao. *Vision Transformer Adapter for Dense Predictions*. URL: https://arxiv.org/pdf/2205.08534.

[11] Pytorch. *AdamW*. URL: https://docs.pytorch.org/docs/stable/generated/torch.optim.AdamW.html.

[12] Pytorch. *OneCycleLR*. URL: https://docs.pytorch.org/docs/stable/generated/torch.optim.lr_scheduler.OneCycleLR.html.

[13] Pytorch. *MultiStepLR*. URL: https://docs.pytorch.org/docs/stable/generated/torch.optim.lr_scheduler.MultiStepLR.html.

[14] Vishal Rajput. *Robustness of different loss functions and their impact on network's learning capability*. URL: https://arxiv.org/pdf/2110.08322.

[15] Wikipedia community. *Early stopping*. URL: https://en.wikipedia.org/wiki/Early_stopping.

[16] Jonathan Bgn. *Simple Audio Augmentation with PyTorch*. URL: https://jonathanbgn.com/2021/08/30/audio-augmentation.html.

[17] Buslaev, Iglovikov, Khvedchenya, Parinov, Druzhinin, Kalinin. *Albumentations: Fast and Flexible Image Augmentations. Information*. URL: https://doi.org/10.3390/info11020125.

[18] C Kishor Kumar Reddy et al. "A fine-tuned vision transformer based enhanced multi-class brain tumor classification using MRI scan imagery". In: *Frontiers in oncology* 14 (2024), p. 1400341.

[19] Junwei Liang et al. "Multi-dataset training of transformers for robust action recognition". In: *Advances in Neural Information Processing Systems* 35 (2022), pp. 14475–14488.

[20] Streamlit Community. *Streamlit documentation*. URL: https://docs.streamlit.io/.

[21] LarxeL. *COVID-19 CT scans*. URL: https://www.kaggle.com/datasets/andrewmvd/covid19-ct-scans.

[22] Piyush Samant DS. *Lung CT nodule/ Lesion Segmenbtation*. URL: https://www.kaggle.com/datasets/andrewmvd/covid19-ct-scans.

[23] Pytorch Community. *Pytorch utils data*. URL: https://docs.pytorch.org/docs/stable/data.html#torch.utils.data.ConcatDataset.

[24] Pytorch Community - Michael Carilli. *Automatic Mixed Precision*. URL: https://docs.pytorch.org/tutorials/recipes/recipes/amp_recipe.html.

[25] Proyash Paban Sarma Borah. *Understanding IoU, Precision, Recall, and mAP for Object Detection Models*. URL: https://medium.com/@Spritan/understanding-iou-precision-recall-and-map-for-object-detection-models-4f06511d289c.

[26] Dongqi Yang, Binqing Xiao, Mengya Cao, Huaqi Shen. *A new hybrid credit scoring ensemble model with feature enhancement and soft voting weight optimization*. URL: https://www.sciencedirect.com/science/article/abs/pii/S0957417423026039#preview-section-cited-by.

[27] mKhaiTruong. *Thesis-Project*$_{Improving - Pneumonia - Detection - with - U - net}$. URL: https://github.com/mKhaiTruong/Thesis-Project_Improving-Pneumonia-Detection-with-U-net.

[28] Seaborn Community. *seaborn.violinplot*. URL: https://seaborn.pydata.org/generated/seaborn.violinplot.html.